AI Models and Applications report: Face Recognition

Arno Lesage (no. 22202985)[a]

[a]Université Côte d'Azur EUR - DS4H - Master 1 Informatique, parcours IA

Face recognition is a classic computer vision application used to test experimental models and for privacy applications, including identification and surveillance. While computing the similarity between faces may seem straightforward, mapping these faces to their owners' names is more challenging. In this report, we investigate multiple processing pipelines to achieve this goal using different convolutional neural networks.

Face-Recognition | CNN | VGG Face | Preprocessing

Face recognition is a computer vision application that aims to match a person's face to their name. The objective of this project is to develop a processing pipeline that can recognise celebrities in different contexts, given a dataset of celebrities. In Section 1, we describe the dataset used for this study. Section 2 deals with our methodology for conducting this study and overcoming the challenges we faced. Section 3 shows the results obtained. Finally, the last two sections encompass conclusions, perspectives, and author contributions.

1. Dataset

This section describes and explores the dataset used for this study.

Initially, the dataset chosen for this study was VGG Face Parkhi et al. (2015a). However, due to unavailable download links, our attention turned to the Pins Face Recognition dataset Burak (2019). This dataset is composed of 17,534 faces of 105 different celebrities, crawled from Pinterest before 2019, and is licensed under CC0 Public Domain.

Each cropped picture of a celebrity's face displays various properties, including different picture sizes, face inclination, colour scale (mainly RGB and grey), luminosity, accessories (e.g. glasses or necklaces) and context. This variety of pictures ensures the model's scalability to larger instances.

With regard to possible intrinsic biases in the dataset, we observe a highly unequal distribution of pictures among celebrities, ranging from 86 pictures (0.49% of the total dataset) for Lionel Messi to 237 pictures (1.35% of the total dataset) for Leonardo DiCaprio. This could potentially lead to imbalanced learning. Additionally, while the balance between male and female celebrities appears to be respected (47.6% and 52.4% of the classes, respectively), there is a significant ethnic disparity in terms of skin colour, with only two black men (1.9% of the classes) represented. This observation may corroborate the difficulty of segregating and classifying black people, especially black-skinned women, if they were present in the dataset. We could also discuss the representation of age, accessories and other topics, but it would be difficult to analyse all of these factors based on the images alone.

2. Methodology

As explained in the introduction, our goal is to find the correct celebrity name associated with a picture of that celebrity, given that picture. More formally:

Goal. Let $(p, c) \in \mathbb{P} \times \mathbb{C}$ be the association of a picture p in the set of all the pictures, $\mathbb{P}$, in the aforementioned dataset, to its corresponding celebrity c in the set of all celebrities, $\mathbb{C}$, within the same dataset, obtained by manual annotation through an unknown function $f : \mathbb{P} \to \mathbb{C}$. We want to create a function $f' : \mathbb{P} \to \mathbb{C}$ such that the number of differences between $f(p)$ and $f'(p)$ is minimised, for all $p \in \mathbb{P}$.

Considering our goal, the backbone to achieve such a function could be explained through a preprocessing pipeline, which is initially expected to be the following:

1. Use YOLO Jocher and Qiu (2024) to extract people from an image,
2. Extract the faces of people identified in the previous step using `RetinaFace` Serengil and Ozpinar (2024),
3. Use the pre-trained `VGG Face Descriptor` model Parkhi et al. (2015b) to identify the celebrity corresponding to the extracted face.

Unfortunately, as with every other study, this one had to overcome multiple difficulties, which we will discuss in the next subsections, dealing with each part of the above preprocessing pipeline.

A. YOLO: How to extract people from a picture. YOLO (You Only Look Once) is a real-time object detection and image segmentation model developed by Ultralytics. There are multiple versions of YOLO; this study uses the `YOLO11n` version, which is pre-trained on the COCO dataset citelin2015microsoft and can recognise 80 classes of object, including people.

Initially thought to be used on the VGG Face dataset featuring uncropped images of people, the goal of this step was to extract a sub-image of persons within a defined image in order to proceed a step further to the extraction of faces.

Unfortunately, using the Pins Face Recognition dataset, which already features cropped images, meant that this preprocessing step was detrimental. More precisely, we observed that `YOLO11n` was not performative enough to categorise faces as belonging to people, which led to cut-off faces, false positives and incorrect classifications.

As this preprocessing step altered the Pins Face Recognition dataset in a harmful way, it was ultimately decided to skip this step.

B. `RetinaFace`: How to extract faces from a picture. `RetinaFace` is a *deep learning based cutting-edge facial detector for Python coming with facial landmarks.*

Although this powerful face detection model was originally thought to function after the preceding preprocessing step, it was ultimately used as the first preprocessing step.

Regarding the relevance of retaining this step despite the images in the Pins Face Recognition dataset already being cropped to the face. We observe that using `RetinaFace` produces more concentrated images of faces, whereas YOLO destroys the faces themselves.

This model also comes with interesting features, such as automatic bounding box extraction and face alignment, which corrects the inclination of faces.

Unfortunately, extracting faces using this model took a very long time for a preprocessing step (between 15 and 20 hours using a CPU). While the quality and associated features may still justify the use of such a model, the waiting time involved makes typical "trial and error" approaches impractical.

This is why we moved to a `RetinaFace` wrapper: `batch-face` Zheng (2025), which allows us to treat picture batches instead of processing them one at a time. This reduces the computing time to under 20 minutes, but the downside is that face alignment is no longer supported, and that face extraction must be done manually after the model detects faces.

Remark 2.1. If no faces are detected in a picture, we consider it to be of insufficient quality for our model and therefore drop it.

C. `VGG Face Descriptor`: How to map pictures to people's names. The `VGG Face Descriptor` is a pre-trained model on the VGG Face dataset that precisely meets our goal of providing a function $f' : \mathbb{P} \to \mathbb{C}$ such that the number of differences between $f(p)$ and $f'(p)$ is minimised for all $p \in \mathbb{P}$. The model is available here in three formats: MatConvNet, Torch (Lua) and Caffe.

Unfortunately, some modifications are required to use the model. Despite being available in Torch, the model uses an old version of Lua Torch that is only compatible with older versions of both Python 2 and PyTorch. This makes the integration process difficult without disrupting the entire pipeline, even in a controlled virtual environment.

One possible solution would be to use software such as `convertTorch2PyTorch` or `torchfile` to convert the Lua Torch into PyTorch externally. Clcarwin (2017), Shillingford (2016). Sadly, these tools are practically as old as our model and therefore suffer from the same compatibility issues.

Another possible approach is to build the model from scratch. Fortunately, this approach is simplified by the Tensorflow-Keras Python library, which implement the VGG Face Predictor architecture (VGG16) Simonyan and Zisserman (2015) whilst not being pre-trained for our purposes. There are two main solutions from here:

1. Train the model from scratch using the Pins Face Recognition dataset, which has the advantage of being fully focused on our objectives. However, it was found to be quite lengthy when training was stopped after 20 hours without achieving satisfactory results.
2. Find the weights and labels of the `VGG Face Descriptor` model and use them in the VGG16 architecture.

Ultimately, the second option was made possible thanks to the files `rcmalli_vggface_tf_vgg16.h5` (weights) and `rcmalli_vggface_labels_v1.npy` (labels), which are available on GitHub in the Python library: Keras-VGGFace Malli (2017).

Nevertheless, when using the second option, we encountered a final issue: the celebrities in the VGG Face dataset, on which our model is pre-trained, do not match those in the Pins Face Recognition dataset. Consequently, we need to filter out every celebrity from the Pins Face Recognition dataset that is not present in the VGG Face dataset. Ultimately, only 37 of the 105 celebrities are present in both datasets, reducing our dataset from 17,500 pictures to 6,131 pictures (34 faces were not detected and have already been filtered out). Consequently, using this model reduces the predictive scale that could have been achieved by training it from scratch, but also reduces the inference time for known celebrities, thus exposing the **training-time vs predictive-scale** trade-off of this study. Additionally, since the pre-trained model can predict a celebrity out of the Pins Face Recognition dataset when fed an image from the aforementioned dataset, a new class `[NR]` must be added (Not Recognised). This class is automatically attributed during inference whenever the model recognises something outside the Pins Face Recognition dataset.

Another alternative would be to train a much simpler CNN using Tensorflow. This approach drastically reduces training time (although it keeps it high), but also leads to reduced predictive power.

D. Additional preprocessing. Apart from the preceding points, other preprocessing steps are applied to improve the final accuracy, or simply to enable the models to work, or to work faster.

The first step is to resize all the images to 224x224 pixels (the default size for the VGG Face model) after `RetinaFace`. This is achieved using the Python library PIL and the `Image.resize` function with the default nearest neighbour interpolation method.

The second step, which is also carried out by PIL, involves converting the image from RGB to greyscale. This is done for two reasons:

- This reduces the training and inference time of the custom model.
- This could potentially improve accuracy by specialising the models in one colour scale instead of multiple ones.

The third step is to load the images in batches of 100 instead of one at a time, which would drastically increase training and inference time, or all at once, which would saturate the memory in the case of 17,500 pictures. Images are only loaded in a batch when necessary, so as not to saturate the overall memory.

Nevertheless, an important preprocessing step is the formation of the training, validation and test sets. There are two cases here:

1. Once the model has been trained, simply put all the images in the test set. We just need to make inferences.
2. If the model needs to be trained, form the training set as a proportion of randomly chosen instances (here, 80% or 75% of the whole dataset) and use the training set for validation purposes.

Remark 2.2. It is important to note that when we say "randomly", we are not referring to uniform randomness across the entire dataset, but rather to uniform randomness within each dataset class, thus achieving a weighted uniform randomness across the entire dataset. This approach was chosen to address the disparities in the number of instances per class observed in Section 1. By doing so, we ensure that approximately 80% of instances from each class are effectively used for training, rather than potentially none from smaller classes if uniform randomness were applied to the entire dataset.

Remark 2.3. Other preprocessing steps that were not tested but are worth mentioning are luminosity equalisation to normalise the range of luminosity among all the pictures, removing the luminosity factor from the learning process, similar to the reason of converting RGB to greyscale. Also worth mentioning is Principal Component Analysis (PCA), which compresses the images in memory.

E. Summarised pipeline. Now that all the challenges have been outlined in the preceding subsections, the global pipeline used for this study is presented below:

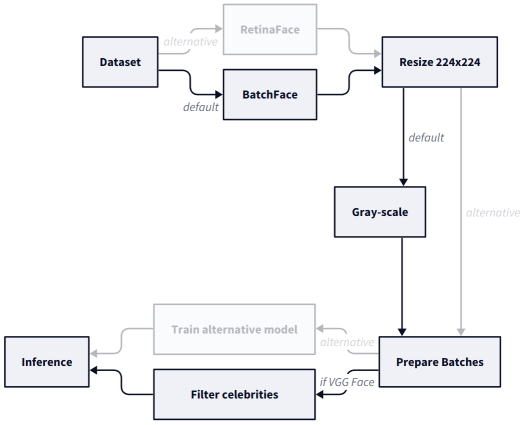

Fig. 1. The global processing pipeline for the study

Although the above Figure 1 illustrates the overall process of this study, it is important to understand that it is flexible and subject to change. In Section 3, we will explain each processing step for all the models presented.

3. Results

In this section, we will describe the various models built, as well as their components. These models will ultimately be assessed and compared collectively. The assessment will focus on various metrics, including training and inference time, accuracy and the number of supported celebrities, as well as other parameters on a case-by-case basis.

A. Models description and initial results. Below is a brief description of each model used and its corresponding pipeline:

Scratch VGG Face Descriptor. The first model attempted in this project is a simple pipeline with the aim of training the VGG16 Face Descriptor model architecture from scratch. To achieve this, we strictly followed the instructions provided for this project, proceeding with the following pipeline:

1. Use `YOLO11n` to extract the persons,
2. Use `Retina-Face` to extract a person's face,
3. Use `additional preprocessing` to normalise the pixel colour range from ⟦0,255⟧ to [0,1],
4. Use the `Custom VGG16 CNN` to predict the person in the picture. It is adapted using *nn.AdaptiveAvgPool2d((7, 7))* to allow for pictures of different sizes as input.

This model most closely follows the initial instructions, but is also the least practical. Despite stating in the methodology that `YOLO11n` is not a good choice, we use it here, as well as `Retina-Face` instead of `Batch-Face`. This results in a very long preprocessing time of between 15 and 20 hours. Additionally, training the CNN itself takes a very long time, with training of this model being aborted after 20 hours and 20 epochs with no convincing results. Despite trying to "prune" the model architecture to reduce training time, this did not change the outcome.

Simple Grey-CNN. The second model was inspired by the initial concept of reducing the training time of the preceding model by pruning the number of layers in VGG16. The idea was to speed up the training process by adding further pre-processing and using a very simple CNN instead of the VGG16 model itself. Ultimately, our CNN used eight layers instead of 23. Here is this model's pipeline:

1. Use the `Batch-Face` tool to rapidly extract faces and filter images where faces are not recognised,
2. Use `additional preprocessing` to resize images to 224x224 pixels, convert them to greyscale, normalise the pixel colour range from ⟦0,255⟧ to [0,1], and organise the pictures into batches,
3. Predict the person in the picture using a `simple CNN`.

This is the most general working model that we have created, in the sense that it can recognise all the celebrities in our dataset. Unfortunately, it has several drawbacks, including a long training time of approximately 16 hours, and low accuracy and weighted accuracy of 17% and 15%, respectively, on the test set.

Nevertheless, it is interesting to note that this model can identify information in these pictures, enabling greater accuracy than a fully random model.

The confusion matrix for this model is shown below:

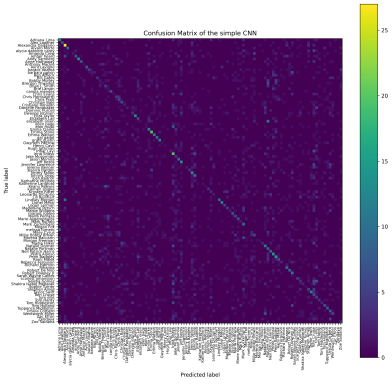

Fig. 2. Confusion Matrix of the Simple Grey-CNN

Pre-trained VGG Face Descriptor. Later on, the weights of the original VGG Face Descriptor model were found on Malli (2017) GitHub repository. We decided to use these weights to infer the names of the celebrities in our pictures directly. The pipeline used in this case is shown below:

1. Use the `Batch-Face` tool to rapidly extract faces and filter images where faces are not recognised,
2. Use `additional preprocessing` to resize images to 224x224 pixels, convert them to greyscale (for greyscale version), normalise the pixel colour range from ⟦0,255⟧ to [0,1], and organise the pictures into batches,
3. Predict the person in the picture using a `pre-trained VGG Face Descriptor` model, a VGG16 model with specialised weight to predict the name of a celebrity in a given picture.

We finally achieved good accuracy levels of above 64% (above 75% for weighted accuracy) with these models, but at the cost of model scalability. In fact, the model can no longer recognise all 105 celebrities in our dataset, only a small fraction of them: 37 celebrities.

The confusion matrices for these models are shown below:

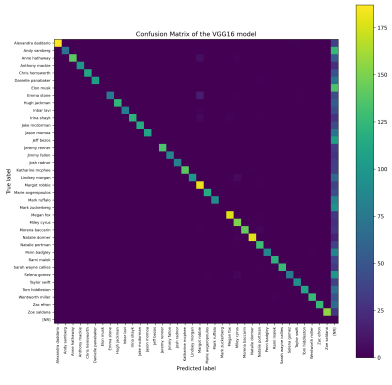

Fig. 3. Confusion Matrix of the pre-trained VGG Face Descriptor [Grey]

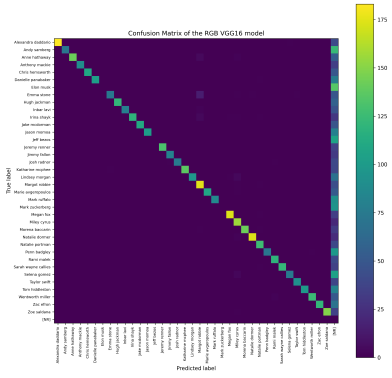

Fig. 4. Confusion Matrix of the pre-trained VGG Face Descriptor [RGB]

Remark 3.1. In these confusion matrices, we can see that the diagonal is well populated and that there was almost no confusion between the 37 celebrities in the subset, which is a very positive sign. However, we also see a line appearing on the right, corresponding to the aforementioned celebrities being misclassified as other celebrities included in the VGG Face dataset but not the Pins Face Recognition dataset. Some celebrities, such as Elon Musk and Mark Zuckerberg, are simply not recognised for some reason, but without access to VGG Face, we cannot explain why.

B. Comparison of the models.

Now that we have looked at all the models we will use, we want to compare them. You will find the details for all the models and their respective pipelines in Table 1.

Training time. This table reveals several interesting findings. Firstly, in terms of training time, we can see that the `Scratch VGG Face Descriptor` model takes the longest to train, at a minimum of 20 hours, just to train the CNN (with inconclusive results). Add to this the inference time of the *Retina-Face* model for preprocessing the data, which adds a further 15 to 20 hours to the full pipeline training time. As mentioned, this long training time for the full pipeline is due to two factors: the *Retina-Face* preprocessing step, which does not support batches, and the VGG16 model itself, which has approximately 145 million trainable parameters, making any attempt at fast training using a CPU futile. The second-longest model to train is, of course, the `Simple Grey-CNN` model, taking approximately 16 hours to train the CNN itself and nine minutes for the pre-processing steps using *Batch-Face*. Despite the number of parameters falling to around 67,000, training the CNN remains time-consuming on a CPU. However, unlike the previous model, this one achieved 200 epochs (compared to 20 for the first model) and was therefore able to produce some results. Finally, the other pre-trained models do not require training, making them ready to use and faster to set up.

In the end, the following partial order is established for the full training pipeline:

$$\texttt{Scratch VGG Face Descriptor}\ [+35\text{h},\ 8.55\ \text{s/picture}] \preceq$$
$$\texttt{Simple Grey-CNN}\ [\sim 16\text{h},\ 4.13\ \text{s/picture}] \preceq$$
$$\texttt{Pre-trained VGG Face Descriptor (Grey, RGB)}\ [0\text{h}]$$

Inference time. The comparison of inference times is straightforward. Firstly, no inferences were made for the `Scratch VGG Face Descriptor` model because training was aborted. For the second model, `Simple Grey-CNN` we found that the total inference time for this model was the same as for *Simple CNN*, which equates to 32 seconds (we had already included *Batch-Face [Grey]* in the training context), making it the fastest model for inference. This could be explained by the low number of parameters. Finally, the last two models have almost identical inference times: 20 minutes in total, including pre-processing (which was not performed in the training context as it is absent for these models), and 11 minutes for inferences only.

Overall, the following partial order applies to the full infer-

ence pipeline:

`Scratch VGG Face Descriptor` [Unknown] $\preceq$
`Pre-trained VGG [...]` [20 min, 0.14 s/picture] $\preceq$
`Simple Grey-CNN` [32s, 0.009 s/picture]

Accuracy. We made the following observations regarding the accuracies. First, disregarding the `Scratch VGG Face Descriptor` model, for which no inferences were made, we observe that the `Simple Grey-CNN` model achieves the lowest accuracy, with weighted and unweighted accuracies of 0.15 and 0.17, respectively. Although this accuracy is fragile, it is not disastrous; indeed, it is far better than random classification over 105 classes. This can be explained by the CNN architecture used, which, despite being small, can still capture some patterns in the pictures. The second-worst accuracy is achieved by the `Pre-trained VGG Face Descriptor [RGB]` model, with a weighted accuracy of 0.75 and an accuracy of 0.64. This very high accuracy can be directly explained by the weights being pre-trained on a very similar dataset. The loss compared to perfect accuracy can be explained by the introduction of foreign labels, which create more possibilities for mismatch. The best accuracy is achieved by the `Pre-trained VGG Face Descriptor [Grey]` model, with an accuracy of 0.66 and a weighted accuracy of 0.77. The explanation for this accuracy is similar to the previous one, but we can also consider that a slight improvement can be attributed to the change in colour scale, which seems to have concentrated the information in a context where changes in luminosity and colour are highly variable from picture to picture.

In the end, we have the following partial order on accuracies:

`Scratch VGG Face Descriptor` [Unknown] $\preceq$
`Simple Grey-CNN` [0.17, w: 0.15] $\preceq$
`Pre-trained VGG Face Descriptor (RGB)` [0.64, w: 0.75] $\preceq$
`Pre-trained VGG Face Descriptor (Grey)` [0.66, w: 0.77]

Remark 3.2. Here, the difference in range between the accuracy and weighted accuracy confirms the imbalance in our data that we saw earlier, and suggests that this imbalance is probably dominant in our subset of 37 celebrities.

Number of supported classes. We will not go into much detail on this matter, but in our case, only two models are capable of recognising all the actors: the `Scratch VGG Face Descriptor` and the `Simple Grey-CNN` models. The others are not capable of that because the pre-trained weights they use do not include those in our dataset. This makes the first two "broader".

How to choose a model?. Finally, we might have to make a choice supported by the aforementioned metrics between our models.

If we wanted to build on a ready-to-use model, we would choose the one with the highest accuracy and no required training, which would result in choosing the `Pre-trained VGG Face Descriptor [Grey]` model. This model would also have been the predominant choice if accuracy had been considered important, especially in contexts where fewer classes need to be predicted.

In another context where inference time must be as fast as possible, we would recommend the `Simple Grey-CNN` model directly. This model exhibits very fast inference, but at the cost of long training times and poor accuracy. This model would also have been chosen if broad inferences were important.

Of the models tested, two stand out: `Simple Grey-CNN` due to its fast inferences on all celebrities, and `Pre-trained VGG Face Descriptor [Grey]` due to its lack of training and higher accuracy.

Table 1. Models components comparative table

Model	Training Size	Training Time	Inference Size	Inference Time	Accuracy	Class qte.	Parameters
Scratch VGG Face Descriptor	~13151	+20 hours	N/A	+15 hours	N/A	105	N/A
YOLO11n	N/A	N/A	17534	20 minutes	N/A	N/A	batchSize: 15
Retina-Face [RGB]	N/A	N/A	~17534	15 to 20 hours	N/A	N/A	N/A
VGG16 (custom)	~13151 (75% per celebrity)	+20 hours (aborted)	N/A	N/A	N/A	105	batchSize: 8 epochs: 20 optimizer: SGD loss: CrossEntropy learningRate 0.005 momentum: 0.9 train/test split seed: 0
Simple Grey-CNN	14038	~16 hours	3462	~10 minutes	0.17 (w: 0.15)	105	N/A
Batch-Face [Grey]	N/A	N/A	17534	9 minutes	N/A	N/A	batchSize: 100 threshold: 0.95
Simple CNN	14038 (80% per celebrity)	16 hours	3462 (20% per celebrity)	32 seconds	0.17 (w: 0.15)	105	batchSize: 100 epochs: 200 optimizer: Adam loss: CrossEntropy train/test split seed: 0
Pre-trained VGG Face Descriptor [Grey]	N/A	N/A	6131	20 minutes	0.66 (w: 0.77)	37	N/A
Batch-Face [Grey]	N/A	N/A	17534	9 minutes	N/A	N/A	batchSize: 100 threshold: 0.95
Pre-trained VGG Face Descriptor [Grey]	N/A	N/A	6131	11 minutes	0.66 (w: 0.77)	37	weights: rcmalli_ vggface_tf_vgg16.h5 batchSize: 100
Pre-trained VGG Face Descriptor [RGB]	N/A	N/A	6131	20 minutes	0.64 (w: 0.75)	37	N/A
Batch-Face [RGB]	N/A	N/A	17534	9 minutes	N/A	N/A	batchSize: 100 threshold: 0.95
Pre-trained VGG Face Descriptor [RGB]	N/A	N/A	6131	11 minutes	0.64 (w: 0.75)	37	weights: rcmalli_ vggface_tf_vgg16.h5 batchSize: 100

Remark 3.3. The inference time refer to the total amount of time passed on inferences **in the pipeline**. Therefore, inferences of *Batch-Face* can be interpreted as inference time in a training context for preprocessing. The same remark apply for the training time.

Conclusion and perspectives

In this work, we explored the problem of face recognition, focusing specifically on how to associate a face with the name of the corresponding person. In the Section 1, we discussed the Pins Face Recognition dataset used in this project Burak (2019), its characteristics, and its limitations.

Then, in a second section, we set out and explained the pipelines we were going to use, ranging from a simple pipeline using *YOLO* Jocher and Qiu (2024), *Retina-Face*, *VGG* Parkhi et al. (2015b) to those incorporating additional preprocessing steps, *YOLO* removal, *Batch-Face* usage and different predictive model testing.

Finally, in a third section, we compared the different pipelines, defining their strengths and weaknesses in terms of accuracy and training/inference times, as well as the number of classes supported by each model.

We showed that our choice of model depends heavily on our application and the surrounding context. Can we train for a long time? Is our system real-time? Do we care about achieving high accuracy? And so on.

In terms of areas for improvement, one possible improvement could be to reduce the training time of our models by using the GPU for computations instead of the CPU, which would greatly increase the possibilities for parallelism. Another possibility would be to work with additional preprocessing steps, such as PCA to concentrate information, a light equaliser to attenuate differences in light between pictures, transfer to another colour scale, or simply face alignment to remove the importance of face angle in our model. However, this approach could only be effective with untrained weights, since we do not know what preprocessing steps were used for the pre-trained weights. Finally, we could perform data augmentation on our dataset to eliminate the bias detected in this report.

Author's contributions

Author's contributions. The contributors to this project are: Arno Lesage (no. 22202985, LinkedIn, ORCID) and Minh Quan Hoang (no. hm509118, LinkedIn)

The first contributor is Arno Lesage, who made the following contributions: literature research, coding, conducting experiments, writing reports, interpreting results and delivering the final presentation. Following his request, the second contributor's name was removed from this report. However, I would like to acknowledge his contribution, especially in terms of the literature research, at the very beginning of the project.

Author's presentations. My name is Arno Lesage and I am a first-year Master's student on the AI track of the Computer Science programme at Université Côte d'Azur. I graduated with a BEng in Data Science from IUT Côte d'Azur, and spent my final bachelor's year abroad studying for a BSc in AI on the Computer Science and Data Science track at Johannes Kepler Universität Linz. During my internships at the i3S/CNRS laboratories, I had the opportunity to experience a research environment and publish a paper in Symbolic AI.

Following this and my bachelor's thesis at the Institut für Analysis on a probabilistic dynamical system, I am now interested in pursuing a PhD, probably in the symbolic side of AI and the study of dynamical systems more broadly.

References

Burak (2019). Pins face recognition. https://www.kaggle.com/datasets/hereisburak/pins-face-recognition.

Clcarwin (2017). converttorch2pytorch. https://github.com/clcarwin/convert_torch_to_pytorch.

Jocher, G. and Qiu, J. (2024). Ultralytics yolo11. https://github.com/ultralytics/ultralytics.

Malli, R. C. (2017). Keras vgg face. https://github.com/rcmalli/keras-vggface/releases/.

Parkhi, O. M., Vedaldi, A., and Zisserman, A. (2015a). Deep face recognition. In *BMVC*.

Parkhi, O. M., Vedaldi, A., and Zisserman, A. (2015b). VGG Face Descriptor. https://www.robots.ox.ac.uk/~vgg/software/vgg_face.

Serengil, S. and Ozpinar, A. (2024). A benchmark of facial recognition pipelines and co-usability performances of modules. *Journal of Information Technologies*, 17(2):95–107.

Shillingford, B. (2016). Torchfile. https://github.com/bshillingford/python-torchfile.

Simonyan, K. and Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition.

Zheng, E. (2025). batch-face. https://github.com/elliottzheng/batch-face.